# 프로젝트 소개와 명령어 지도

발표자: 이지영 · 코드 설명을 포함한 상세 대본 · 시연 포함 약 12~15분

## 발표 시작

안녕하세요. Python으로 CLI 기반 Mini Git을 만든 이지영입니다. 이번 프로젝트는 Git의 커밋과 브랜치를 자료 구조로 이해하고, 검색과 그래프 탐색 알고리즘을 직접 구현하는 것이 목적입니다.

오늘은 제가 어떤 명령어를 사용했는지, 그 명령이 코드의 어느 부분으로 연결되는지, 내부 데이터가 어떻게 바뀌는지 설명하겠습니다.

## 명령어별 역할

```
INIT Jiyoung                # 저장소와 작성자 초기화
BRANCH feature              # 현재 위치에 브랜치 생성
SWITCH feature              # 작업 브랜치 선택
COMMIT "로그인 기능 추가"      # 커밋 메타데이터 생성
LOG                         # 부모 우선 로그
LOG --sort-by=date          # 날짜순 정렬
LOG --sort-by=author        # 작성자순 정렬
SEARCH "로그인"              # 키워드 검색
SEARCH --author=Jiyoung     # 작성자 검색
ANCESTORS <해시>             # 모든 조상 조회
PATH <해시1> <해시2>          # 최단 경로 조회
HELP / QUIT                 # 도움말 / 종료
```

이 명령들은 제가 만든 Mini Git에서 사용하는 명령입니다. COMMIT을 실행하면 파일 내용 대신 메시지, 작성자, 시간, 부모 관계를 메모리에 기록합니다. 종료하면 이 기록은 사라집니다.

코드 블록의 # 뒤는 설명입니다. 시연할 때는 명령 부분만 입력합니다. 화면과 시간은 환경에 따라 달라집니다.

# 입력이 코드로 전달되는 과정

## 실행과 REPL

```
python3 main.py
```

일반 터미널에서 이 명령으로 프로그램을 실행합니다. main 함수는 MiniGit 객체를 한 개 만들고, 반복문에서 사용자 입력을 기다립니다.

```
repo = MiniGit()
while True:
    # 입력·출력 부분 발췌
    line = input('mini-git> ')
    output = repo.execute(line)
```

input은 한 줄을 문자열로 읽습니다. execute는 그 문자열을 해석하고 결과 문자열을 반환합니다. main은 반환값을 출력하고 다시 입력을 기다립니다. 이 흐름을 REPL이라고 합니다.

## 명령어와 인자 분리

```
parts = shlex.split(line)
command, *args = parts
command = command.upper()
```

COMMIT "첫 번째 커밋"을 입력하면 command에는 COMMIT이, args에는 메시지 한 개가 들어갑니다. shlex.split을 사용했기 때문에 따옴표 안의 공백을 유지할 수 있습니다.

별표가 붙은 args는 나머지 항목을 리스트로 받습니다. upper는 명령어만 대문자로 바꾸므로 init과 INIT을 같은 명령으로 처리합니다.

이후 허용된 인자 개수를 검사합니다. 예를 들어 INIT은 사용자명 한 개가 필요하고 PATH는 해시 두 개가 필요합니다. 잘못된 입력은 Invalid args로 안내하고 다음 입력을 받습니다.

# INIT과 저장소의 내부 상태

[시연] 새 세션에서 INIT Jiyoung을 입력합니다.

```
INIT Jiyoung
```

## INIT 처리 코드 발췌

```
if self.user is not None:
    raise ValueError('Repository already initialized')
self.user = args[0]
self.branches['main'] = None
```

self는 현재 MiniGit 객체를 의미합니다. self.user에는 앞으로 커밋을 작성할 사용자 이름인 Jiyoung을 저장합니다.

branches는 브랜치 이름과 마지막 커밋 해시를 연결하는 딕셔너리입니다. 초기 main 값이 None인 이유는 아직 커밋이 없기 때문입니다. 현재 브랜치 이름인 self.branch는 객체 생성 시 main으로 설정됩니다.

```
self.user             # 'Jiyoung'
self.branch            # 'main'
self.branches           # {'main': None}
self.commits            # {}
```

초기화가 성공하면 Initialized repository와 현재 브랜치, 사용자 이름이 출력됩니다. 초기화 전에 COMMIT을 실행하면 Repository not initialized가 나옵니다. INIT을 반복해 기존 기록을 지우지 않도록 재초기화도 막았습니다.

## 커밋 객체의 구성

```
@dataclass(frozen=True)
class Commit:
    hash: str
    message: str
    author: str
    timestamp: datetime
    parents: tuple[str, ...]
```

dataclass는 데이터를 담는 클래스를 간결하게 작성하게 해 줍니다. frozen=True로 생성 후 필드 변경을 막았습니다. parents는 부모 해시의 튜플이며, 첫 커밋은 빈 튜플입니다.

# COMMIT: 새 기록과 부모 연결

[시연] COMMIT "첫 번째 커밋"을 입력합니다.

## 커밋 생성 코드 발췌

```
self.counter += 1
h = f'{self.counter:016x}'
head = self.branches[self.branch]
parents = (head,) if head is not None else ()
c = Commit(h, args[0], self.user,
           datetime.now(timezone.utc), parents)
self.commits[h] = c
```

카운터를 1 증가시키고 16진수로 바꿉니다. 016x는 최소 16자리로 표현하며 빈 자리를 0으로 채운다는 뜻입니다. 첫 해시는 0000000000000001이 됩니다.

현재 브랜치의 HEAD가 있으면 그 커밋을 부모로 저장합니다. (head,)의 쉼표는 원소 한 개짜리 튜플이라는 뜻입니다. 첫 커밋처럼 HEAD가 없으면 부모는 빈 튜플입니다.

시간은 UTC로 기록하고, 커밋 객체는 해시를 키로 하는 commits 사전에 저장합니다. 키로 직접 찾기 때문에 평균 조회 시간은 O(1)입니다.

## 경로 탐색용 연결과 HEAD 갱신

```
self.adjacency[h] = set(parents)
for parent in parents:
    self.adjacency[parent].add(h)
self.branches[self.branch] = h
self.index.add(c)
```

커밋 자체는 부모 방향만 저장합니다. adjacency에는 부모와 자식을 양쪽으로 기록하여 PATH에서 무방향 탐색이 가능하게 합니다. 이후 현재 브랜치의 HEAD를 새 해시로 바꾸고 검색 인덱스도 갱신합니다.

정상 CLI에서는 새 커밋이 이미 존재하는 부모만 가리키고 기존 커밋을 수정하지 않습니다. 그래서 커밋 부모 그래프에는 순환이 생기지 않습니다. 이를 DAG라고 합니다.

# BRANCH와 SWITCH: 분기 만들기

[시연] 첫 커밋을 만든 상태에서 아래 순서로 실행합니다.

```
BRANCH feature
SWITCH feature
COMMIT "로그인 기능 추가"
SWITCH main
COMMIT "메인 기능 추가"
```

## 두 명령의 핵심 코드

```
# BRANCH: 현재 커밋 해시를 새 이름에 복사
self.branches[args[0]] = self.branches[self.branch]

# SWITCH: 현재 선택한 브랜치 이름 변경
self.branch = args[0]
```

BRANCH feature는 현재 HEAD 해시를 feature라는 새 이름에 연결합니다. 커밋을 복제하지 않고 동일한 커밋을 가리키게 합니다. 이 명령만으로 작업 브랜치가 바뀌지는 않습니다.

SWITCH feature는 self.branch를 feature로 바꿉니다. 이후 로그인 커밋을 만들면 feature의 HEAD만 두 번째 커밋으로 이동합니다.

다시 main으로 돌아오면 main은 여전히 첫 번째 커밋을 가리킵니다. 여기에서 메인 커밋을 만들면 부모는 첫 번째 커밋이고 main의 HEAD는 세 번째 커밋으로 이동합니다.

```
공통 부모: 0000000000000001
  ├─ feature → 0000000000000002 (로그인 기능 추가)
  └─ main    → 0000000000000003 (메인 기능 추가)
```

따라서 두 번째와 세 번째 커밋은 부모와 자식 관계가 아니라 같은 부모에서 갈라진 커밋입니다. 같은 브랜치 이름을 다시 만들거나 없는 브랜치로 전환하면 오류 메시지를 반환합니다.

# LOG: 부모부터 출력하는 위상 정렬

[시연] LOG를 실행하고 첫 번째 커밋이 앞에 나오는지 보여줍니다.

```
return self.format_commits(
    topological_order(self.commits)
)
```

기본 LOG는 모든 브랜치의 커밋을 대상으로 합니다. topological_order가 해시 순서를 만들고 format_commits가 작성자, 시간, 메시지와 브랜치 표시를 붙입니다.

## Kahn 알고리즘 핵심 발췌

```
degrees = {h: len(c.parents) for h, c in commits.items()}
queue = deque(h for h in commits if degrees[h] == 0)
result = []
while queue:
    h = queue.popleft()
    result.append(h)
    for child in children[h]:
        degrees[child] -= 1
        if degrees[child] == 0:
            queue.append(child)
```

degrees는 아직 처리하지 않은 부모의 수입니다. children은 부모별 자식 목록으로 미리 구성합니다. 부모가 없는 커밋부터 큐에 넣고 popleft로 먼저 들어온 노드를 꺼냅니다.

부모를 출력한 다음 각 자식의 남은 부모 수를 줄입니다. 0이 된 자식만 큐에 들어가므로 모든 부모가 자식보다 먼저 출력됩니다. 예시에서는 첫 커밋을 처리한 뒤 두 번째와 세 번째가 준비됩니다.

모든 노드를 처리하지 못하면 사이클 가능성이 있어 Invalid commit graph 오류를 냅니다. 정상 CLI는 사이클을 만들지 않습니다. 그래프를 한 번씩 처리하므로 시간복잡도는 O(V+E)입니다.

화면의 [main]과 [feature]는 해당 커밋을 현재 가리키는 브랜치입니다. 과거에 어느 브랜치에서 생성됐는지를 고정해서 표시하는 값은 아닙니다.

# PATH: BFS로 최단 경로 찾기

[시연] 두 번째 커밋에서 세 번째 커밋으로 이동합니다.

```
PATH 0000000000000002 0000000000000003
# 결과: 0000000000000002->0000000000000001
#       ->0000000000000003 (실제로는 한 줄 출력)
```

## BFS 핵심 코드 발췌

```
queue = deque([start])
previous = {start: None}
# 반복문에서 도착 여부 검사 후 이웃 탐색
for neighbor in merge_sort(
        list(adjacency[current]), key=lambda h: h):
    if neighbor not in previous:
        previous[neighbor] = current
        queue.append(neighbor)
```

BFS는 가까운 노드부터 단계별로 탐색합니다. 모든 간선의 비용이 같으므로 처음 도착했을 때 간선 수가 가장 적은 경로가 됩니다. previous는 방문 여부와 직전 노드를 함께 기록합니다.

예시에서는 두 번째의 이웃인 첫 번째를 방문하고, 첫 번째를 통해 세 번째를 방문합니다. 도착하면 previous를 거꾸로 따라가 경로를 모은 뒤 뒤집어 출력합니다.

```
path = []
while current is not None:
    path.append(current)
    current = previous[current]
return path[::-1]
```

path[::-1]은 리스트 순서를 뒤집는 표현입니다. 최단 경로가 여러 개면 이웃을 해시 사전순으로 정렬해 탐색하여 요구된 동률 규칙을 적용합니다. 경로가 없으면 No path를 출력합니다.

BFS 자체는 O(V+E)이고, 이 구현은 방문 노드의 이웃을 정렬하는 비용이 추가됩니다. 큐와 방문 기록은 탐색 규모에 따라 커집니다.

# ANCESTORS: DFS로 조상 찾기

[시연] ANCESTORS 0000000000000003을 실행합니다.

```
visited = set()
stack = list(commits[start].parents)
result = []
while stack:
    h = stack.pop()
    if h not in visited:
        visited.add(h)
        result.append(h)
        stack.extend(commits[h].parents)
return result
```

처음에는 조회 대상의 부모만 스택에 넣습니다. 그래서 조회한 커밋 자신은 결과에 포함되지 않습니다. pop은 가장 나중에 넣은 항목을 꺼내므로 깊이를 따라가는 DFS가 됩니다.

아직 방문하지 않은 커밋만 결과에 추가하고 그 부모를 스택에 넣습니다. 여러 경로가 같은 조상에 도달해도 visited 집합 덕분에 한 번만 출력합니다.

세 번째 커밋의 부모는 첫 번째 커밋입니다. 첫 번째는 부모가 없으므로 탐색이 끝납니다. 두 번째 커밋은 형제 위치에 있으므로 조상이 아닙니다.

## PATH와 ANCESTORS의 차이

PATH는 부모와 자식 양쪽으로 움직여 두 점 사이의 최소 거리를 구합니다. ANCESTORS는 부모 방향으로만 움직여 모든 조상을 구합니다. 그래서 서로 다른 연결 자료를 사용합니다.

반복문과 스택을 사용해 재귀 깊이 제한을 피했습니다. 출력 순서는 DFS에서 발견한 순서이며 최신순이나 거리순은 아닙니다. 도달 가능한 그래프 기준 시간은 O(V+E)입니다.

# SEARCH: 역색인으로 후보 가져오기

[시연] SEARCH "로그인"과 SEARCH --author=Jiyoung을 실행합니다.

## 커밋 생성 시 인덱스를 준비하는 코드

```
for token in dict.fromkeys(commit.message.lower().split()):
    self.keywords.setdefault(token, []).append(commit.hash)
self.authors.setdefault(commit.author, []).append(commit.hash)
```

lower는 소문자 변환, split은 공백 기준 분리입니다. dict.fromkeys는 반복 토큰을 제거합니다. setdefault는 키가 없으면 빈 리스트를 만든 뒤 해당 목록을 반환합니다. 여기에 커밋 해시를 추가합니다.

```
# 예시의 인덱스 일부: 해시는 끝자리만 표기
'로그인' → [2]
'기능'   → [2, 3]
'추가'   → [2, 3]
작성자 'Jiyoung' → [1, 2, 3]
```

따라서 로그인 검색은 전체 커밋을 순회하지 않고 로그인 게시 목록을 읽어 한 개를 반환합니다. 작성자 검색은 작성자 사전에서 세 개를 가져옵니다. 실제 인덱스에는 전체 해시가 저장됩니다.

## 작성자 옵션과 여러 단어 검색

```
by_author = query.startswith('--author=')
# 작성자 옵션이면 접두사를 제거한 이름으로 조회
found = self.index.search(query, author=by_author)
```

작성자 이름은 대소문자까지 정확히 일치시킵니다. 여러 키워드는 각각의 게시 목록을 가져와 집합으로 만들고 가장 짧은 목록을 기준으로 모든 목록에 있는 해시만 반환합니다. 이것이 AND 검색입니다.

부분 문자열 검색은 하지 않고 구두점도 유지합니다. 단일 토큰 검색은 평균 O(1)로 목록을 찾고 결과 k개를 복사하는 데 O(k)가 듭니다. 흔한 단어는 결과가 많아 여전히 느릴 수 있습니다.

# 정렬 LOG: 비교 기준과 병합 정렬

[시연] LOG --sort-by=date와 LOG --sort-by=author를 실행합니다.

## 정렬 기준 선택 코드

```
field = 'timestamp' if args[0].endswith('date') else 'author'
ordered = merge_sort(
    list(self.commits.values()),
    key=lambda c: getattr(c, field)
)
```

date 옵션이면 timestamp, author 옵션이면 author 필드를 기준으로 정렬합니다. lambda는 짧은 익명 함수이고 getattr는 이름으로 객체의 속성을 읽습니다. 따라서 정렬 함수 하나에 비교 기준만 바꿔 전달할 수 있습니다.

## 안정성을 결정하는 병합 코드

```
if key(left[i]) <= key(right[j]):
    merged.append(left[i])
    i += 1
else:
    merged.append(right[j])
    j += 1
```

정렬된 왼쪽과 오른쪽 묶음의 앞 원소를 비교하여 작은 값을 먼저 넣습니다. 동률일 때 왼쪽을 먼저 넣기 때문에 원래 순서가 유지됩니다. 이를 안정 정렬이라고 합니다.

실제 구현은 묶음 크기를 1, 2, 4 순서로 늘리는 반복형 병합 정렬입니다. 각 단계가 O(n)이고 단계 수가 O(log n)이므로 키 계산과 비교가 상수일 때 평균과 최악 모두 O(n log n)입니다. 보조 공간은 O(n)입니다.

현재 작성자가 모두 Jiyoung이므로 작성자 정렬 결과는 생성 순서를 유지합니다. 기본 LOG는 부모 우선이지만 정렬 옵션을 주면 지정한 기준을 우선합니다. Python 기본 정렬 API는 사용하지 않았습니다.

# 도움말·오류 처리·종료와 검증

## HELP, EXIT, QUIT

```
if command in ('EXIT', 'QUIT'):
    if args:
        raise ValueError('Invalid args')
    return 'Bye!'

# main 함수에서 종료 판단
if output == 'Bye!':
    break
```

HELP는 사용 가능한 명령 형식을 보여줍니다. EXIT와 QUIT은 Bye!라는 결과를 반환하고 main이 그 값을 확인해 반복문을 종료합니다. Ctrl+C나 입력 종료도 처리합니다.

현재 저장소는 메모리 객체이므로 프로그램을 종료하면 기록이 사라집니다. Python을 실행하는 일반 터미널과 mini-git 프롬프트를 구분해야 합니다. pwd나 실제 git push는 QUIT 후 일반 터미널에서 실행합니다.

## 오류 처리

```
except ValueError as error:
    # 따옴표 파싱 오류는 Invalid args로 통일
    return str(error)
```

위 코드는 예외 처리 구조를 설명한 발췌입니다. 실제 코드는 따옴표 오류 문구를 검사해 Invalid args를 반환합니다.

잘못된 인자, 없는 브랜치, 없는 커밋을 검사하고 안내 문자열로 반환합니다. 예외 때문에 REPL 전체가 종료되지 않아 사용자가 바로 다음 명령을 입력할 수 있습니다.

## 검증 방법

```
python3 -m unittest discover -s tests -v
```

이 명령은 일반 터미널에서 실행합니다. 자동 테스트 6개가 모두 통과했고, 브랜치, 검색, 정렬 안정성, 최단 경로 동률, 조상 중복 제거 등을 확인했습니다. 커밋 1,000개의 해시 유일성과 긴 이력 탐색도 검증했습니다.

# 확장성 설계와 발표 마무리

## 현재 구현의 한계

모든 커밋과 인덱스를 메모리에 보관하므로 데이터가 많아지면 RAM 사용량이 커집니다. 흔한 단어의 게시 목록을 복사하거나 전체 커밋을 정렬하고 출력하는 작업도 병목이 될 수 있습니다.

## 개선 방향

우선 정수 ID와 게시 목록 압축으로 메모리 사용을 줄이고, 필요하면 디스크 기반 저장소를 도입하는 방향을 정리했습니다. 전체 출력은 페이지나 스트리밍 방식으로 나누고 반복 조회에는 캐시를 검토할 수 있습니다.

규모가 더 커지면 키워드와 작성자별 인덱스를 여러 샤드로 나눌 수 있습니다. 하지만 흔한 키워드 한 개가 특정 샤드에 집중될 수 있으므로 큰 목록을 커밋 ID 범위로 추가 분할하는 대책이 필요합니다.

커밋 원본과 인덱스 갱신을 분리하면 반영 지연이나 부분 실패도 생깁니다. 갱신 기록을 남기고 같은 작업을 다시 실행해도 중복되지 않도록 멱등 처리를 설계해야 합니다. 검색 인덱스 분산만으로 그래프 탐색이 빨라지는 것은 아니므로 각각의 병목을 구분해야 합니다.

이 내용은 README에 정리한 향후 설계안입니다. 현재 분산 저장이나 외부 인덱스를 구현한 것은 아닙니다. 실제 성능 측정 후 단계적으로 개선하려는 방향입니다.

## 마무리 발언

이번 프로젝트에서는 명령 입력부터 자료구조 변경과 결과 출력까지 직접 구현했습니다. 커밋은 DAG와 해시맵, 로그는 위상 정렬, 최단 경로는 BFS, 조상 탐색은 DFS, 검색은 역색인, 정렬은 안정 병합 정렬로 연결했습니다.

알고리즘의 이름을 아는 것에서 나아가, 어떤 요구사항에 왜 선택했는지 코드와 실행 결과로 설명할 수 있게 된 것이 이번 프로젝트의 핵심 학습 성과입니다. 이상으로 발표를 마치겠습니다. 감사합니다.

선택 보너스인 Diff, Merge, 정렬 성능 비교는 구현 범위에서 제외했습니다.